FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING

Father Agnel Ashram, Bandstand, Bandra (West), Mumbai – 400050

DEPARTMENT OF COMPUTER ENGINEERING

PROJECT REPORT

ON

GREENCITY – CLOUD-BASED SMART WASTE MANAGEMENT AND COMPLAINT TRACKING SYSTEM

Submitted in partial fulfilment of the requirements for the Cloud Computing Mini Project

Submitted by

SAKSHI

Roll No.: 10895

Academic Year 2026–2027

ABSTRACT

GreenCity is a web-based waste management and complaint tracking system designed to support the digital management of waste-related activities. The application provides an administrative interface through which authorised users can review complaints, manage collection vehicles, maintain waste collection records, view collection locations and monitor operational information through a dashboard.

The local application is developed using a React-based frontend, a Node.js and Express backend, and a MySQL relational database. The system follows a client-server architecture in which the frontend communicates with backend API routes, while the backend performs validation, business logic and database operations.

The project also studies a cloud deployment architecture using Amazon Web Services. The proposed architecture includes a custom Virtual Private Cloud, public and private subnets, an application server on EC2, Amazon RDS, Amazon S3, AWS Systems Manager Parameter Store, IAM roles and Amazon CloudWatch. AWS deployment is pending until it is successfully deployed and verified in the institution-provided AWS Academy environment.

TABLE OF CONTENTS

1. Introduction

2. Problem Definition

3. Objectives and Scope

4. Existing and Proposed System

5. Feasibility Study

6. Software and Hardware Requirements

7. Technologies Used

8. Cloud Computing Concepts

9. System Architecture

10. System Design

11. Database Design

12. Application Modules

13. Frontend Implementation

14. Backend Implementation

15. Security Considerations

16. AWS Cloud Architecture

17. Implementation and Execution

18. Testing and Validation

19. Results and Discussion

20. Limitations

21. Future Enhancements

22. Conclusion

23. References

24. Appendix

Update page numbers after opening the document in Microsoft Word. To create an automatic table of contents, use References → Table of Contents after confirming the heading styles.

CHAPTER 1: INTRODUCTION

1.1 Background

Waste management includes the collection, transportation, processing and disposal of waste. Efficient management requires accurate records, coordination and timely handling of complaints. Manual record keeping can make these activities difficult to monitor.

1.2 Project Overview

GreenCity is a web-based administrative system that brings waste-management activities into one application. It is intended to help an administrator manage complaints, vehicles, collection records and locations, and view operational summaries.

1.3 Motivation

The project is motivated by the need for organised digital records and better visibility of waste-management activities.

CHAPTER 2: PROBLEM DEFINITION

2.1 Problem Statement

In a manual or disconnected process, complaint details, vehicle information and collection records may be stored in different places. This can make tracking, updating and reporting time-consuming. GreenCity addresses this through a central web-based interface.

2.2 Need for the System

Centralised storage of waste-management information.

A structured way to record and track complaints.

Easier maintenance of vehicle and collection details.

A dashboard for operational summaries.

A foundation for cloud hosting.

CHAPTER 3: OBJECTIVES AND SCOPE

3.1 Objectives

Develop a responsive waste-management web application.

Provide authenticated access to administrative functions.

Support complaint, vehicle and collection records.

Display collection locations where configured.

Use a relational database.

Prepare a cloud deployment architecture.

3.2 Scope

The current scope focuses on the administrative web application and its core records. Citizen registration, live GPS tracking, route optimisation and production deployment are not claimed unless separately implemented and tested.

CHAPTER 4: EXISTING AND PROPOSED SYSTEM

4.1 Existing System

Manual processes may use registers, spreadsheets, phone calls or separate applications. Retrieving historical records and preparing summaries can require additional effort.

4.2 Proposed System

GreenCity provides a central administrative interface backed by a server and relational database. The administrator can access available modules from a browser and perform supported record operations.

4.3 Advantages

Centralised information management.

Browser-based access.

Easier retrieval and updating of records.

Reduced dependence on scattered records.

Extendable architecture.

CHAPTER 5: FEASIBILITY STUDY

5.1 Technical Feasibility

React, Node.js, Express and MySQL are commonly available technologies that can run on a development computer and can be hosted using cloud infrastructure.

5.2 Economic Feasibility

Development can use open-source software. Cloud costs depend on selected resources, region, runtime and institutional credits. Actual costs must be checked in the authorised account.

5.3 Operational Feasibility

The browser-based interface is intended to be straightforward for an administrator with basic computer skills.

CHAPTER 6: SOFTWARE AND HARDWARE REQUIREMENTS

6.1 Software Requirements

Operating System: Windows 10/11 or compatible.

Code Editor: Visual Studio Code.

Frontend: React and Vite.

Backend: Node.js and Express.js.

Database: MySQL 8.

Browser: Google Chrome or another modern browser.

Planned cloud platform: Amazon Web Services.

6.2 Hardware Requirements

Computer capable of running a modern browser and development tools.

Internet connection for package installation and cloud access.

Sufficient memory and storage for the development environment.

CHAPTER 7: TECHNOLOGIES USED

7.1 React and Vite

React is used to build the interface as reusable components. Vite provides the frontend development and build environment.

7.2 Node.js and Express

Node.js runs the server-side JavaScript application. Express defines HTTP routes and middleware for API requests.

7.3 MySQL

MySQL is a relational database used to store structured application data. SQL queries are used by the backend to access records.

7.4 Supporting Libraries

The project may use libraries for authentication, password hashing, database connectivity, charts, maps and file handling. Confirm exact dependencies from the package files.

CHAPTER 8: CLOUD COMPUTING CONCEPTS

8.1 Cloud Computing

Cloud computing provides on-demand access to computing, storage, networking and other IT resources over a network.

8.2 Service Models

Infrastructure as a Service provides virtual machines, storage and networking. Platform as a Service provides managed application platforms. Software as a Service provides complete applications.

8.3 Deployment Models

Public cloud services are operated by a cloud provider. Private cloud infrastructure is dedicated to an organisation. Hybrid cloud combines public cloud with private or on-premises infrastructure.

8.4 Benefits

Scalability according to demand.

Managed database and storage options.

Centralised monitoring and logging.

Flexible resource provisioning.

Identity and access management services.

CHAPTER 9: SYSTEM ARCHITECTURE

9.1 Application Architecture

GreenCity follows a three-tier structure: the browser-based React presentation tier, the Node.js and Express application tier, and the MySQL data tier.

9.2 Request Flow

Administrator opens the application in a browser.

Frontend displays the selected module.

Frontend sends an HTTP request to the backend API.

Backend validates the request and performs business logic.

Backend accesses MySQL when required.

Backend returns a response and the frontend updates the display.

9.3 Architecture Diagram

Insert a labelled diagram: Administrator Browser → React Frontend → Express/Node.js API → MySQL Database. Show proposed AWS components separately and label them pending until deployed.

CHAPTER 10: SYSTEM DESIGN

10.1 Functional Requirements

Administrator login and protected pages.

Complaint record management, where implemented.

Vehicle record management, where implemented.

Waste collection record management, where implemented.

Collection locations and map, where configured.

Dashboard summaries and reports, where implemented.

10.2 Non-Functional Requirements

Usability: consistent and understandable pages.

Performance: reasonable response under tested workload.

Security: protected operations require authentication.

Maintainability: separate frontend, backend and database responsibilities.

Reliability: errors handled without exposing confidential data.

10.3 Use Case Description

The primary actor is the Administrator. The administrator logs in and uses the available dashboard, complaint, vehicle, collection, location, report and settings functions. Include only use cases present in the final application.

CHAPTER 11: DATABASE DESIGN

11.1 Database Overview

The application uses MySQL as its relational database. The development database has previously been identified as greencity; verify this against the active configuration before submission.

11.2 Main Entities

Users: account and authentication information.

Complaints: complaint details and status.

Vehicles: vehicle-related information.

Waste Collections: collection activity records.

Collection Locations: location information.

11.3 Relationships

Use the actual SQL schema to document column names, primary keys and foreign keys. Insert the verified ER diagram here; do not assume relationships that are absent from the schema.

11.4 Integrity

Use primary keys to identify records and foreign keys where implemented. Validate required fields before database operations.

CHAPTER 12: APPLICATION MODULES

12.1 Admin Login

The login module provides an entry point for authorised administrators. The backend should verify submitted credentials and return an authentication result.

12.2 Dashboard

The dashboard presents a summary of information available in the application. Add a screenshot and describe only metrics actually displayed.

12.3 Complaint Management

The complaint module maintains complaint records. Describe the actual view, add, update, delete and status functions available.

12.4 Vehicle Management

The vehicle module maintains vehicle-related records. Include exact fields and actions visible in the final interface.

12.5 Waste Collection Management

This module maintains waste collection records. Include only fields present in the database and interface.

12.6 Collection Locations

The location module presents collection-location information. If a map is enabled, insert a screenshot of the actual map.

12.7 Reports and Settings

Describe only the charts, filters and profile or settings options present in the completed application.

CHAPTER 13: FRONTEND IMPLEMENTATION

13.1 Frontend Structure

The frontend uses React and Vite. The interface is divided into components and pages so that individual modules can be maintained separately.

13.2 User Interface

The interface is designed for browser-based administration. Insert genuine screenshots of the running login page, dashboard and completed modules.

13.3 API Communication

The frontend sends HTTP requests to backend API routes. Responses are used to display records, confirmations and errors.

13.4 Responsive Design

Test the layout at desktop and smaller screen widths. Report only responsive behaviour that has been verified.

CHAPTER 14: BACKEND IMPLEMENTATION

14.1 Backend Structure

The backend uses Node.js and Express. It receives requests, applies middleware and routes, performs business logic and accesses MySQL.

14.2 API Operations

List the actual API methods and paths after checking the backend route files.

14.3 Database Connectivity

The backend uses a MySQL driver to connect to the database. Connection values should be loaded from protected configuration and not hard-coded.

14.4 Error Handling

The backend should return suitable HTTP status codes and readable errors without exposing sensitive stack traces or credentials.

CHAPTER 15: SECURITY CONSIDERATIONS

15.1 Authentication

Protected routes should require authentication. Confirm the actual login and token/session implementation in the backend.

15.2 Password Protection

Passwords should be stored using a suitable one-way password-hashing function, not plain text. Confirm the actual algorithm and settings before documenting them.

15.3 SQL Safety

Use parameterised queries or prepared statements to reduce SQL injection risk.

15.4 Secret Management

Do not commit credentials or include them in screenshots. In the proposed AWS environment, Systems Manager Parameter Store can hold configuration values, with access granted through an IAM role.

15.5 Access Control

Use IAM roles and least-privilege policies for cloud resources.

CHAPTER 16: AWS CLOUD ARCHITECTURE

16.1 Proposed Design

The planned architecture uses a custom Amazon VPC with public and private subnets. EC2 hosts the backend, Amazon RDS hosts the relational database, and Amazon S3 provides object storage if required by the application.

16.2 Networking

The VPC design includes subnet associations, route tables, an Internet Gateway for required public connectivity and security groups restricting traffic. The database should remain in private subnets.

16.3 Compute and Database

EC2 is proposed for the application backend and RDS for MySQL. The application server should connect to the database over the private network using restricted security-group rules.

16.4 Storage and Configuration

S3 can store application files. Systems Manager Parameter Store can provide configuration at runtime. IAM instance roles should be used instead of hard-coded AWS access keys.

16.5 Monitoring and Cost

CloudWatch can collect logs and metrics and support alarms. AWS Budgets or the budget facility available in the lab account can monitor estimated spending.

16.6 Deployment Status

AWS deployment is pending until resources and the application are successfully deployed and verified in the institution-provided AWS Academy environment. Configuration files or templates alone do not prove deployment.

CHAPTER 17: IMPLEMENTATION AND EXECUTION

17.1 Local Setup

The local environment uses a React/Vite frontend, Node.js/Express backend and MySQL. Start the database, configure local environment variables, then start the backend and frontend.

17.2 Backend Execution

Open PowerShell in the backend directory and run the configured start command, such as `node server.js` if that is the entry point. Confirm the server listening message.

17.3 Frontend Execution

Open a second terminal in the frontend directory and run the configured development script, commonly `npm run dev`. Open the local URL printed by Vite.

17.4 Evidence

Insert genuine screenshots of the terminal, browser and database. Hide passwords, tokens, connection strings and other secrets.

CHAPTER 18: TESTING AND VALIDATION

18.1 Testing Approach

Test login, page loading, form validation, database operations and API responses. Test each implemented module with valid and invalid input.

18.2 Suggested Test Cases

Login with valid administrator credentials.

Login with an incorrect password.

Open a protected page without authentication.

Create a record with valid required fields.

Submit a form with missing required fields.

Update an existing record.

Delete a record where supported.

Refresh and verify saved data.

18.3 Test Results

Record the actual result after executing each test. Do not report a test as passed unless it has been run and observed. Include genuine test logs or screenshots if available.

CHAPTER 19: RESULTS AND DISCUSSION

19.1 Expected Outcome

The completed local application should provide a central interface for the administrative functions that have been implemented and connected to the database.

19.2 Observed Outcome

Add a factual description based on the final running application. Include screenshots and mention incomplete modules. Do not describe proposed cloud resources as deployed.

19.3 Discussion

The project demonstrates separation of frontend, backend and database responsibilities and defines a cloud architecture for future hosting.

CHAPTER 20: LIMITATIONS

20.1 Current Limitations

Cloud deployment remains pending until verified in the institutional AWS environment.

Features absent from the source code or interface are outside the implemented scope.

Live GPS tracking and route optimisation are not claimed unless tested.

Performance and security have not been independently audited unless formal tests were conducted.

20.2 Constraints

Deployment depends on AWS Academy access, lab permissions, service quotas and the permitted budget.

CHAPTER 21: FUTURE ENHANCEMENTS

21.1 Possible Improvements

Citizen registration and complaint submission portal.

SMS or email notifications.

Live GPS tracking of collection vehicles.

Route optimisation for collection planning.

Role-based access for different staff members.

Advanced analytics and downloadable reports.

Mobile-friendly citizen interface.

Verified cloud deployment with automated backup and recovery.

CHAPTER 22: CONCLUSION

22.1 Conclusion

GreenCity is a web-based project intended to organise waste-management administration through a central application. Its local architecture uses a frontend, backend and relational database. The project also defines a cloud deployment approach using AWS networking, compute, database, storage, identity and monitoring services. Local implementation and cloud deployment must be evaluated separately; cloud deployment can be claimed as complete only after successful deployment and verification.

22.2 Learning Outcomes

The project provides practical experience in full-stack development, REST API communication, relational databases, application security considerations and cloud architecture planning.

CHAPTER 23: REFERENCES

23.1 Documentation

React: https://react.dev/

Vite: https://vite.dev/

Node.js: https://nodejs.org/docs/

Express: https://expressjs.com/

MySQL: https://dev.mysql.com/doc/

AWS: https://docs.aws.amazon.com/

23.2 Project Sources

Use the project's source code, package configuration, database schema and actual test output as the primary references for implementation-specific details.